

TAKE-HOME

Full-stack take-home: smart grocery list

Hi, and thanks for taking the time. This should be about 90 minutes of work. We are not looking for a

polished product. We want to see how you make decisions, how you handle a vague requirement, and

how you read code that someone else wrote.

Use AI tools. We mean it. Copilot, Claude, Cursor, ChatGPT, whatever you reach for day to day.

We build

with these tools and we want to see you do the same. There is one catch: you have to tell us how you

used them (see AI_LOG.md below). Not using AI is not the smart play here.

There is no live call. We grade the submission on its own. So write things down.

What you are building

A small web app: a smart grocery list.

The user types grocery items into a text box, one per line or one at a time, your choice. Each item gets a

colored tag. The tag color depends on what kind of item it is. You decide the color for a kind by a set of

rules, and you check the rules in order. First rule that matches wins. Stop checking after the first match.

Here are the three rules, in order:

1. If the item has a quantity and a unit in it, like "2 kg rice" or "500 ml milk", parse out the number and

the unit and show a "measured" badge. The badge shade scales with how big the quantity is. A small

amount is a light shade, a large amount is a dark shade. Pick a sensible mapping and a sensible ceiling.

2. Otherwise, if the item is a known pantry staple, show a neutral gray "staple" tag. The staple list is short and hardcoded: salt, sugar, flour, rice, oil. That is the whole list. Do not get clever with it.

3. Otherwise, ask an LLM to classify which grocery aisle the item belongs to. The aisles are:

produce,

dairy, bakery, frozen, household. Color the tag by aisle (you choose the five colors). This rule has to call a real LLM on your backend. Rule 3 is the only rule that is allowed to call the LLM.

That ordering matters. Rule 1 beats rule 2 beats rule 3.

The part we left vague on purpose

Look at "2 kg salt". Salt is a staple (rule 2). But "2 kg salt" also has a quantity and a unit (rule 1). So which tag wins?

We did not tell you. That is the point. Decide what the right behavior is, build that, and explain why in

DECISIONS.md. There is no single correct answer. We want to see you notice the conflict, pick a side,

and defend it in a few sentences. A choice with a clear reason beats a "best" choice with no reasoning.

Hard requirements

These are not optional. Missing one costs real points.

Login

Gate the app behind a login that only allows @company.com email addresses. Anyone else gets bounced. It does not need to be bulletproof auth, but a non-Company address must not get in.

Real LLM on the backend

Rule 3 has to hit a real LLM, and the call has to happen server side, not in the browser. No mocked response, no hardcoded aisle map standing in for the model. If we look at your code we want to see an

actual API call going out from your backend.

Deploy it

Ship it on Vercel. Give us the live URL and a public repo. We are going to open the app and click

around,
so it has to actually run.

Accessibility

Two things, both checked by us in a browser:

The whole app has to be usable with the keyboard alone. Tab to the input, type, submit, move through results, all without a mouse.

The text on every tag has to stay readable as the background color changes. Your measured badge gets darker as quantities grow, so light text on a light badge or dark text on a dark badge is a real risk

here. Measure the actual contrast ratios with a tool and paste the numbers into your README. We want the real measured ratios for your lightest and darkest badge states, not a claim that it "looks fine". WCAG AA for normal text is 4.5 to 1.

Files we want in the repo

A README with the live URL, the repo is already the repo, how to run it locally, and your measured contrast ratios.

A DECISIONS.md. Cover the "2 kg salt" conflict and anything else where you had to choose. Short is fine.

An AI_LOG.md. Tell us which AI tools you used, paste two or three of the prompts that actually moved the work forward, and describe a couple of moments where the AI gave you something wrong and you caught it and fixed it. That last part is the part we care about most. Everyone uses AI. We want to see that you check it.

The reverse code review

In this repo there is a folder called review/ . It has a small Python helper module (quantity_shade.py)

and a test file for it (test_quantity_shade.py). This is the parser that turns "2 kg" into a normalized number and a badge shade, the same idea behind rule 1.

We wrote this code. It has problems in it. Some are bugs, some are not. Your job is to review it like you

would review a coworker's pull request.

Write a REVIEW.md in the repo root. In it:

Point out what is wrong. For each thing, say where it is and why it is a problem.

If something would burn us in production but is not strictly a "bug", flag that too.

Tell us what you would change.

You do not have to fix the code. A clear written review is what we are scoring. Reading the tests as carefully as you read the code is a good idea.

How we score it

Roughly: getting it built and deployed and the LLM call real is the biggest single chunk. Then the accessibility work done for real, the vague-rule decision, and the code review each carry meaningful

weight. The AI log and your overall judgment count too. You pass at 65 out of 100.

You do not need every requirement perfect to pass. A working app with honest notes and a sharp code

review beats a feature-complete app with no reasoning and a review that missed everything.

Good luck. Have fun with it.